Topic 6: Lists

Simplified Lecture Notes

Concepts and definitions only

What Is a List?

- A **list** stores a collection of values in a single variable, kept in order.
- An ordinary variable holds **one** value; a list holds **many**.
- Values inside a list are called **elements**, written inside square brackets `[ ]` and separated by commas.
- The number of elements is the **size** of the list.

List vs Primitive Data Type

- A **primitive** (integer, float, string) holds a **single** value.
- A **list** holds **many** values at once.
- A primitive holds **one** data type; a list can hold **different** data types together.
- A list is written with brackets `[ ]`; a primitive has none.

Data a List Can Store

- Any data type can be stored in a list.
- **Integers**, **floats**, **strings**, and **booleans** are all valid elements.
- A single list may **mix** these types together.
- A list can also be **empty**, ready to be filled later.

Characteristic 1: Ordered

- Elements are stored in a **specific sequence**.
- The order is set when the list is created and is kept.
- Each element has a fixed **position** (its index) in that order.

Characteristic 2: Mutable

- A list can be **changed** after it is created.
- You may **add**, **remove**, or **update** its elements.
- This is what makes a list flexible during a program's run.

Characteristic 3: Allows Duplicates

- The **same value** may appear more than once in a list.
- Duplicate elements are all kept, each at its own position.

Characteristic 4: Different Data Types

- A single list can store integers, strings, floats, and booleans **together**.
- The list does not force every element to be the same type.

Characteristic 5: Dynamic Size

- A list can **grow** or **shrink** as needed.
- Its size is not fixed; elements can be added or removed at any time.

List Indexing

- Each element has a **position**, called its **index**.
- Indexing is how you reach a single element in the list.
- Indexes can be counted from the **front** (positive) or the **back** (negative).

Positive Index

- Indexing starts at **0**, not 1.
- The first element is at index `0`, the second at `1`, and so on, left to right.
- The last element of a list of size *n* is at index *n* − *1*.
- Using an index that does not exist causes an **error**.

Negative Index

- Negative indexes count from the **end** of the list.
- The last element is at index `-1`, the second-to-last at `-2`, right to left.
- They reach the same elements as positive indexes, counted the other way.

Creating a List

- A list can be created **empty**, with nothing inside the brackets.
- A list can also be created **with elements** already in it.
- Both are valid starting points, depending on your need.

Updating a List

- A list is **mutable**, so its elements can be changed after creation.
- To update an element, assign a **new value** to it through its index.
- Updating works with both **positive** and **negative** indexes.

Pre-defined List Functions

- **Python** provides built-in **functions** that give information about a list.
- A function is used by writing its **name** with the list inside the brackets.
- The four covered here are `len()`, `min()`, `max()`, and `sum()`.

The len() Function

- `len()` returns the **number of elements** in a list.
- It can also count characters in a string.
- It is often used to check a list's size before working with it.

The min() Function

- `min()` returns the **smallest** element in a list.
- With numbers, it returns the lowest value.
- With strings, it returns the word **first** in alphabetical order.

The max() Function

- `max()` returns the **largest** element in a list.
- With numbers, it returns the highest value.
- With strings, it returns the word **last** in alphabetical order.

Comparable Types Only

- `min()`, `max()`, and `sum()` must **compare or add** elements.
- **Python** cannot compare a number against text.
- The list must hold a **single comparable type**: all numbers, or all strings.
- Mixing types causes a **TypeError**.

The sum() Function

- `sum()` adds up all the **numeric values** in a list and returns the total.
- An optional **starting value** can be added to the total.
- It works only on numbers, not on strings.

Function vs Method

- A **function** is written as `name(list)`, with the list **inside** the brackets.
- A **method** is written as `list.name()`, attached to the list **after a dot**.
- A method is like a function, but it belongs to an object.

The append() Method

- `append()` adds a **single element** to the **end** of a list.
- It **modifies** the list in place.
- It is often used with an empty list to **build** a list one element at a time.
- Combined with a condition, it can add only elements that qualify.

The sort() Method

- `sort()` rearranges a list's elements into **order**.
- By default it sorts in **ascending** order (smallest to largest).
- Passing `reverse=True` sorts in **descending** order (largest to smallest).
- It orders **strings** alphabetically as well as numbers.

> **6.6** Sorting with `sort()` is covered as an **Assignment** outcome.